Renad Ibrahim

You can access all the plots on the public github repo for failure cases and more experiments https://github.com/renadibrahim23/siemensCapstoneFinal

Siemens Digital Twin Course Winter 2026 Capstone Project

Introduction

This report presents the design, implementation, and evaluation of a modular robotic control system integrated using the Virtual System Interconnect (VSI) framework. The system is composed of three main components: a Simulator, a Controller, and a Logger, each with clearly defined responsibilities and real-time communication over the VSI network.

The Simulator models the robot's kinematics and generates sensor feedback, while the Controller implements a PID-based path-following algorithm with noise filtering and rate-limited actuation to ensure smooth and accurate tracking. The Logger records and visualizes system states, including position, orientation, and control errors, enabling performance analysis and validation of the control strategy.

By leveraging VSI, this architecture allows for seamless integration of distributed components, realistic simulation of control scenarios, and flexibility for future extensions. The report details the system design, control methodology, noise modeling, and integration strategy, demonstrating the system's capability to accurately follow predefined paths under realistic conditions.

Objectives and Scope

The primary objective of this project is to develop and evaluate a modular robotic control system capable of following predefined paths with accuracy and stability, while being fully integrated through the Virtual System Interconnect (VSI) framework. The project aims to demonstrate how distributed components can communicate and function together in real-time, simulating realistic control and sensing scenarios.

The specific objectives include:

1. **Controller Design: Implement a PID-based controller that tracks lateral and angular errors while compensating for sensor noise and limiting actuation rates for smooth motion.**

2. **Simulation of Robot Dynamics: Model the robot's motion kinematics in a simulator component to generate realistic feedback for testing the controller.**
3. **Data Logging and Visualization: Record system states and control outputs to analyze performance metrics and visually compare actual paths with desired trajectories.**
4. **VSI-Based Integration: Enable real-time communication between distributed components (Simulator, Controller, Logger) through VSI, ensuring synchronization and interoperability.**

The scope of the project is limited to path-following scenarios in a 2D environment using predefined trajectories such as straight lines, sine waves, and circular paths. While the system simulates realistic motion and measurement noise, physical hardware implementation is not within the scope of this report. This approach allows for rapid testing, iteration, and evaluation of control strategies in a controlled virtual environment.

3. System Modeling

3.1 Robot Kinematics

The robot is modeled as a differential-drive mobile robot operating in a two-dimensional plane. Its state is defined by its position $(x, y)$and orientation θ.

The motion of the robot is governed by the following kinematic equations:

$$\dot{x} = v\cos(\theta), \dot{y} = v\sin(\theta), \dot{\theta} = \omega$$

where $v$represents the linear velocity and ωrepresents the angular velocity.

In this implementation, the linear velocity is fixed at a constant value of $v = 0.5$, while the angular velocity is controlled by the PID controller to ensure accurate path tracking.

3.2 Path Modeling

The reference trajectory is defined as a function y_{ref}, which varies depending on the selected path type.

For a straight-line path, the reference is constant:

$$y_{ref} = 0$$

For a sinusoidal path, the reference varies with time according to:

$$y_{ref}(t) = 0.5\sin(2\pi \cdot 0.1 \cdot t)$$

This introduces a dynamic trajectory that requires continuous adjustment from the controller.

For the curved path, a sequence of circular arcs is used. Each arc is defined by the equation:

$$y = r - \sqrt{r^2 - x^2}$$

Multiple segments are concatenated to create a continuous path with alternating curvature.

3.3 Error Definition

The main objective of the controller is to minimize the lateral deviation from the path. This is quantified using the lateral error:

$$e(t) = y_{ref} - y_{measured}$$

To simulate realistic sensing conditions, measurement noise is added to the robot's position:

$$y_{noisy} = y_{measured} + \text{noise}$$

The control error is then computed using the noisy measurement:

$$e(t) = y_{ref} - y_{noisy}$$

Low-Pass Filtering

To reduce the effect of noise, a first-order low-pass filter is applied to the error signal:

$$e_f(t) = \alpha e(t) + (1 - \alpha) e_f(t - 1)$$

where $\alpha = 0.2$.
This filtering step smooths the error signal before it is used by the controller.

Heading Error

In addition to lateral error, the heading error is computed to measure the difference between the robot orientation and the desired orientation:

$$e_\theta = \text{atan2}(\sin(\theta_{ref} - \theta), \cos(\theta_{ref} - \theta))$$

In this implementation, the desired orientation is set to $\theta_{ref} = 0$.

3.4 PID Controller Design

The angular velocity command is generated using a PID controller based on the filtered lateral error:

$$\omega = K_p e + K_i \int e\, dt + K_d \frac{de}{dt}$$

The controller is implemented in discrete form. The integral and derivative terms are approximated as:

$$I = I + e \cdot dt$$
$$D = \frac{e - e_{previous}}{dt}$$

The final control signal is then computed as:

$$\omega = K_p e + K_i I + K_d D$$

3.5 Practical Enhancements

To improve stability and robustness, several practical modifications were introduced to the controller:

Integral Anti-Windup

The integral term is limited to prevent excessive accumulation:

- For straight and sinusoidal paths: $I \in [-5,5]$
- For circular paths: $I \in [-1,1]$

This tighter bound for curved paths improves performance under higher curvature.

Deadband Handling

Very small errors are ignored to reduce oscillations:

$$|\, e \,| \leq 0.005 \Rightarrow e = 0$$

Rate Limiting

To ensure smooth steering, the rate of change of angular velocity is limited:

$$\Delta\omega \leq \Delta\omega_{max}$$

where:

- $\Delta\omega_{max} = 0.8$for normal operation
- $\Delta\omega_{max} = 1.2$for curved paths

Saturation

The angular velocity is constrained within a fixed range:

$$\omega \in [-3,3]$$

3.6 Control Pipeline

The overall control process can be summarized as follows:

1. The robot's state $(x, y, \theta)$is measured
2. The reference path $y_{ref}$is computed
3. Noise is added to simulate sensor imperfections
4. The error is filtered using a low-pass filter
5. The PID controller computes the control signal
6. Rate limiting and saturation are applied
7. The final velocity commands $(v, \omega)$are sent to the robot

3.7 Noise and Disturbance Modeling

To better reflect realistic operating conditions, measurement noise is introduced into the system. Instead of using the true robot position directly, noise is added to the measured lateral position before computing the control error.

The noisy measurement is modeled as:

$$y_{noisy} = y_{measured} + n(t)$$

where $n(t)$represents a random noise signal. In this implementation, the noise is generated using a uniform distribution:

$$n(t) \sim \mathcal{U}(-A, A)$$

with a noise amplitude of $A = 0.01$.

This noise directly affects the lateral error calculation:

$$e(t) = y_{ref} - y_{noisy}$$

As a result, the controller operates on a noisy error signal, which introduces high-frequency fluctuations that can degrade performance.

To mitigate this effect, a first-order low-pass filter is applied to the error signal before it is used by the PID controller. This filtering step reduces sensitivity to noise while preserving the overall trend of the error.

The inclusion of noise allows for a more realistic evaluation of the controller, particularly in robustness experiments where performance under disturbances is assessed.

4. Controller Design and Implementation

4.1 Control Objective

The primary objective of the controller is to minimize the lateral deviation of the robot from the reference path while ensuring smooth and stable motion. This is achieved by adjusting the robot's angular velocity based on the tracking error.

A secondary objective is to maintain stable behavior under noise and varying path curvature.

4.2 PID Controller Design

A PID controller is used to compute the angular velocity ωbased on the lateral error. The controller combines three components:

- The proportional term reacts to the current error
- The integral term accounts for accumulated past errors

- The derivative term predicts future error trends

$$\omega = K_p e + K_i \int e\, dt + K_d \frac{de}{dt}$$

The gains K_p, K_i, and $K_d$are configurable parameters and were tuned experimentally.

4.3 Discrete-Time Implementation

The controller operates in discrete time with a fixed timestep $dt = 0.01$seconds.

The integral term is updated as:

$$I = I + e \cdot dt$$

The derivative term is computed as:

$$D = \frac{e - e_{previous}}{dt}$$

The final control signal is then calculated using these components.

4.4 Noise Handling

Since the system operates with noisy measurements, the raw error signal is filtered before being used by the controller. A first-order low-pass filter is applied:

$$e_f = \alpha e + (1 - \alpha) e_f^{previous}$$

This reduces the effect of high-frequency noise and prevents instability caused by rapid fluctuations in the derivative term.

4.5 Anti-Windup Strategy

To prevent integral windup, the accumulated integral error is bounded within predefined limits.

For straight and sinusoidal paths, a wider bound is used to allow steady-state correction. For curved paths, a tighter bound is applied to improve responsiveness and prevent overshoot.

This adaptive strategy improves performance across different path types.

4.6 Rate Limiting

To ensure smooth control actions, the rate of change of angular velocity is limited. Instead of directly applying the PID output, the change in $\omega$is constrained:

$$\Delta\omega = \omega_{target} - \omega_{previous}$$

If $\Delta\omega$exceeds a predefined threshold, it is clipped.

A higher limit is used for curved paths to allow faster turning when required.

This prevents sudden changes in steering, improving stability and realism.

4.7 Output Saturation

The angular velocity is limited within a fixed range:

$$\omega \in [-3,3]$$

This reflects actuator constraints and prevents extreme control commands.

4.8 Constant Linear Velocity

The linear velocity is fixed at:

$$v = 0.5$$

This simplifies the control problem by decoupling speed from steering, allowing the controller to focus entirely on minimizing lateral error.

4.9 Summary

The controller combines a standard PID structure with several practical enhancements, including filtering, anti-windup, rate limiting, and saturation. These additions significantly improve performance, especially in the presence of noise and curved paths.

5.0 VSI / System Integration

The system architecture consists of three VSI clients: **Controller**, **Simulator**, and **Logger**, which interact over the VSI TLM fabric using CAN-like communication. Each client operates independently but synchronizes through the VSI server.

System Architecture

1. **Simulator**
    - Maintains the simulated robot state: x, y, θ.
    - Updates position based on linear velocity (linear_v) and angular velocity (angular_w) received from the Controller.
    - Sends the updated states to the Controller and Logger using dedicated CAN IDs:
        - 10 → pos_x
        - 11 → pos_y
        - 12 → theta
        - 14 → measured_x
        - 15 → measured_y
        - 17 → measured_theta
2. **Controller**
    - Computes control commands based on received measurements (measured_x, measured_y, measured_theta) from the Simulator.
    - Implements PID control with path-specific adjustments:
        - **Low-pass filtering** for noise suppression.
        - **Rate limiting** for angular velocity.
        - **Anti-windup** on integral error.
    - Sends outputs to Simulator and Logger:

        - 13 → angular_w
        - 16 → linear_v
        - 18 → lateral_error
        - 19 → theta_error

3. **Logger**
    - Receives all relevant signals from both Simulator and Controller.
    - Logs signals to a CSV file in real-time and provides live plotting:
        - pos_x, pos_y, theta
        - angular_w, lateral_error, theta_error
        - Computes target path values for comparison (target_y)

Signal Mapping Table

Signal	**Sender**	**Receiver(s)**	**CAN ID**
pos_x	Simulator	Logger, Controller	10
pos_y	Simulator	Logger, Controller	11
theta	Simulator	Logger	12
measured_x	Simulator	Controller	14
measured_y	Simulator	Controller	15
measured_theta	Simulator	Controller	17
angular_w	Controller	Simulator, Logger	13
linear_v	Controller	Simulator, Logger	16
lateral_error	Controller	Logger	18
theta_error	Controller	Logger	19

Data Flow

- The **Simulator** updates the robot's state every simulation step and transmits it.
- The **Controller** receives state measurements, computes control commands, and sends the commands back to the Simulator.
- The **Logger** collects both the measured states and control outputs, logs them for post-analysis, and updates live plots for visual verification.

Noise Modeling

- The Controller simulates sensor noise by adding small random perturbations to the measured y-position.
- A low-pass filter is applied to reduce the effect of high-frequency noise on the control output, improving stability and path-following accuracy.

Advantages of VSI Integration

- **Synchronized simulation** across multiple clients without race conditions.
- **Clear separation of roles**: simulation, control, and logging.
- **Real-time data access** for analysis and debugging.
- **Scalable architecture**: additional modules (e.g., sensor models) can be added without changing existing clients.

6. Experiments and results

Experiment 1: Gain Sweep and Experiment 4: PID vs PD

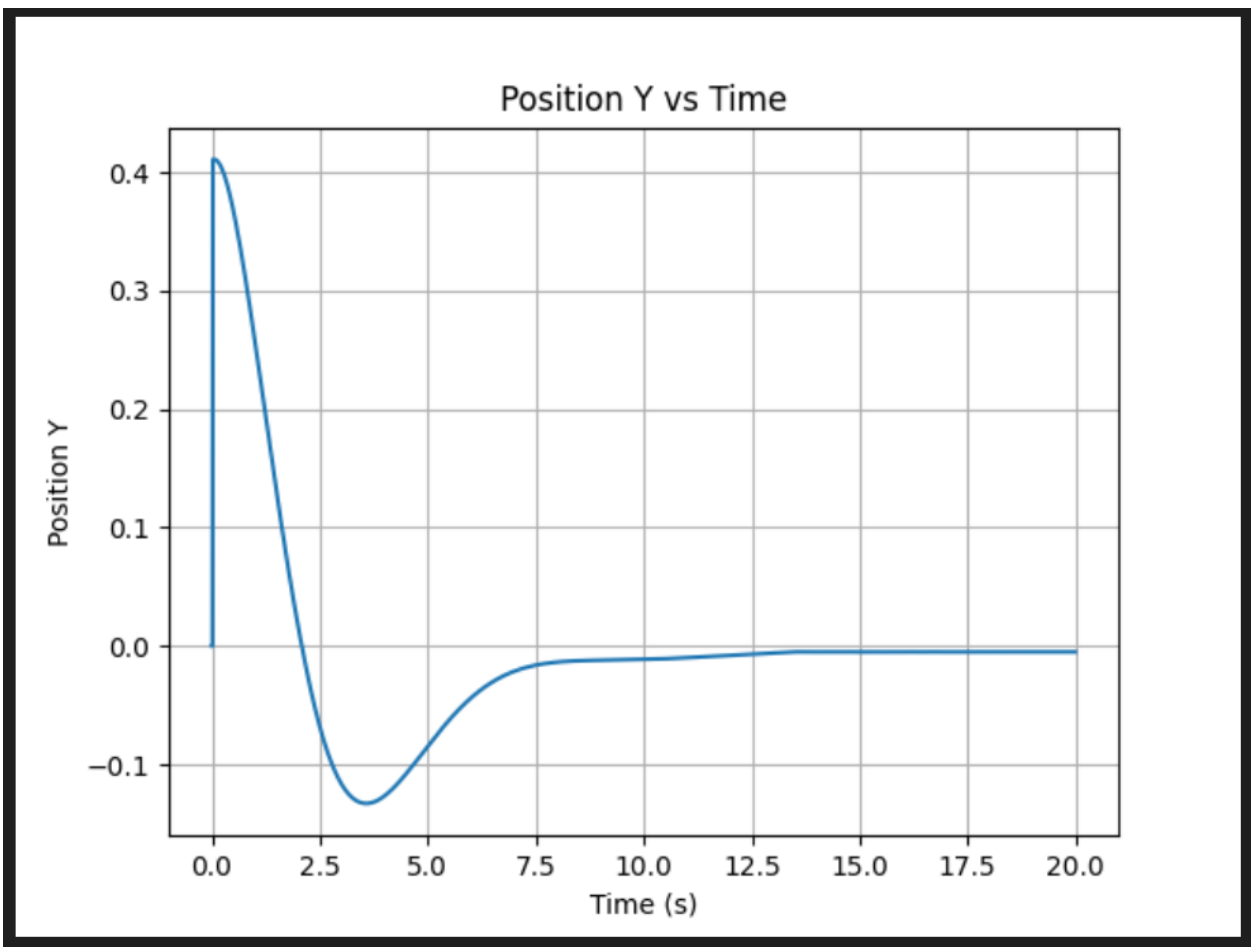
For this experiment I ran the simulator on the straight-line experiment wth 3 different sets of Kp, Ki and Kd using random spawn functionality.

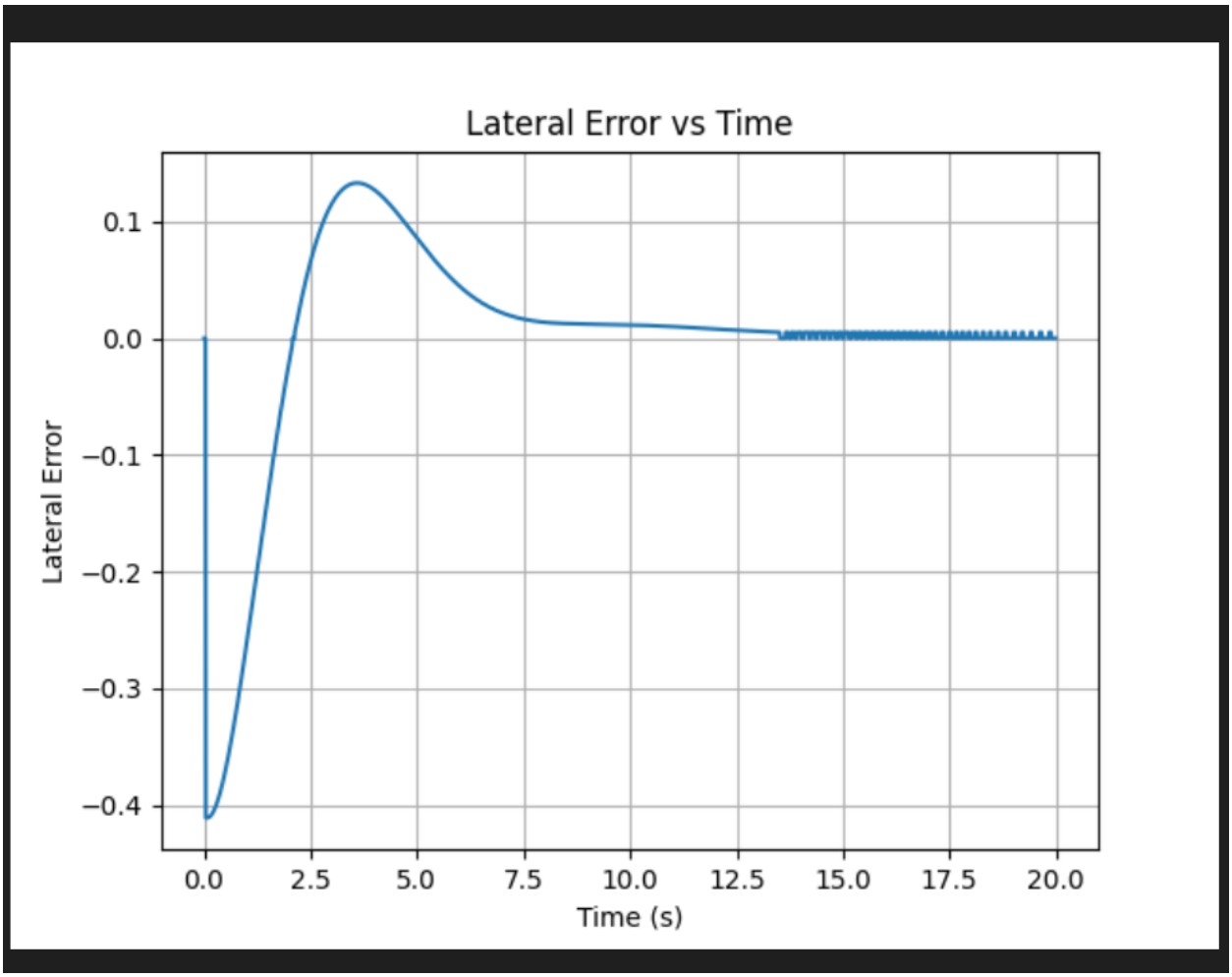
Random Spawn

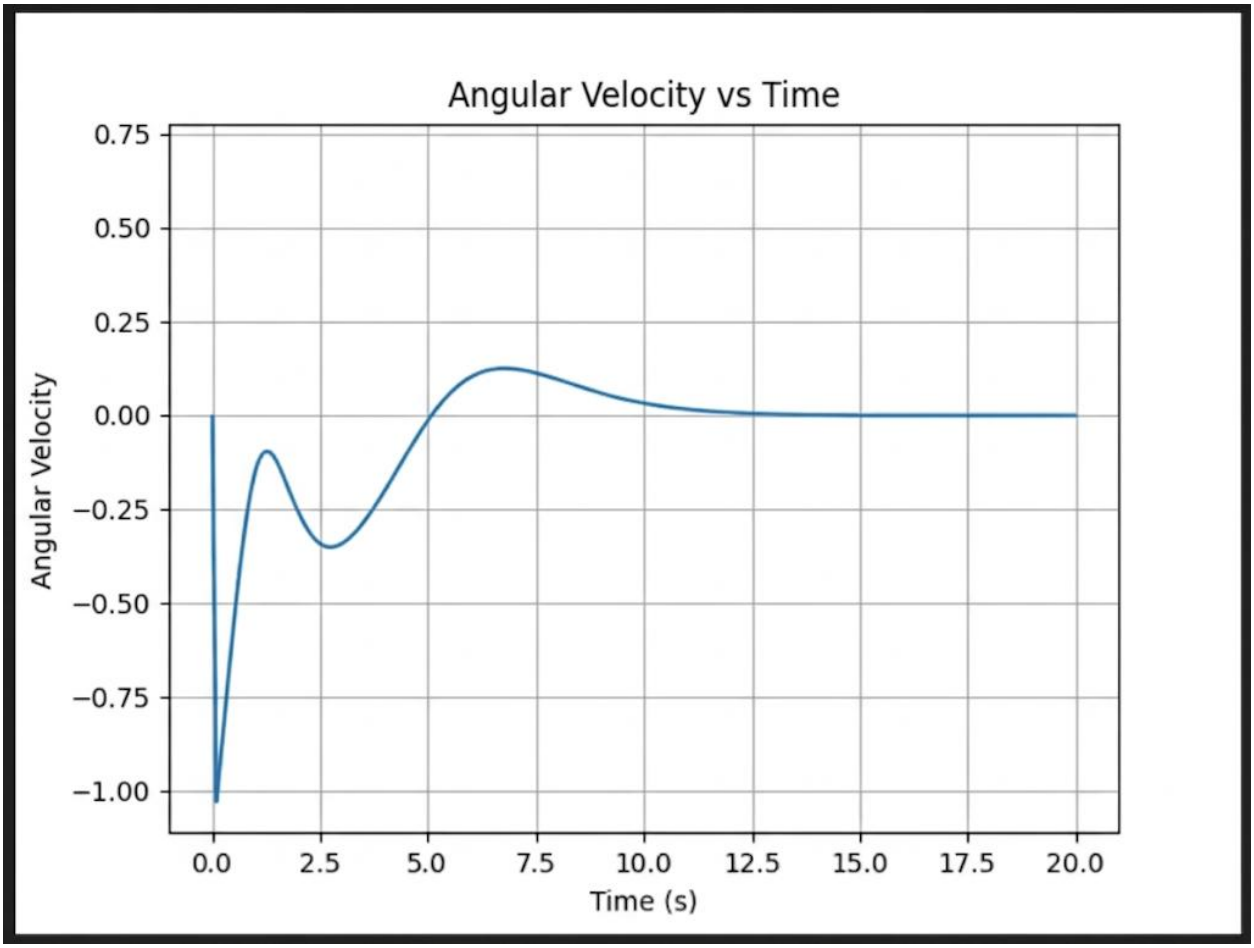
```
if random_spawn:
    self.mySignals.pos_y=random.uniform(-0.5, 0.5)
    print(f"--RANDOM SPAWN: starting at Y= {self.mySignals.pos_y:.3f}---")
else:
    self.mySignals.pos_y=0.5
```

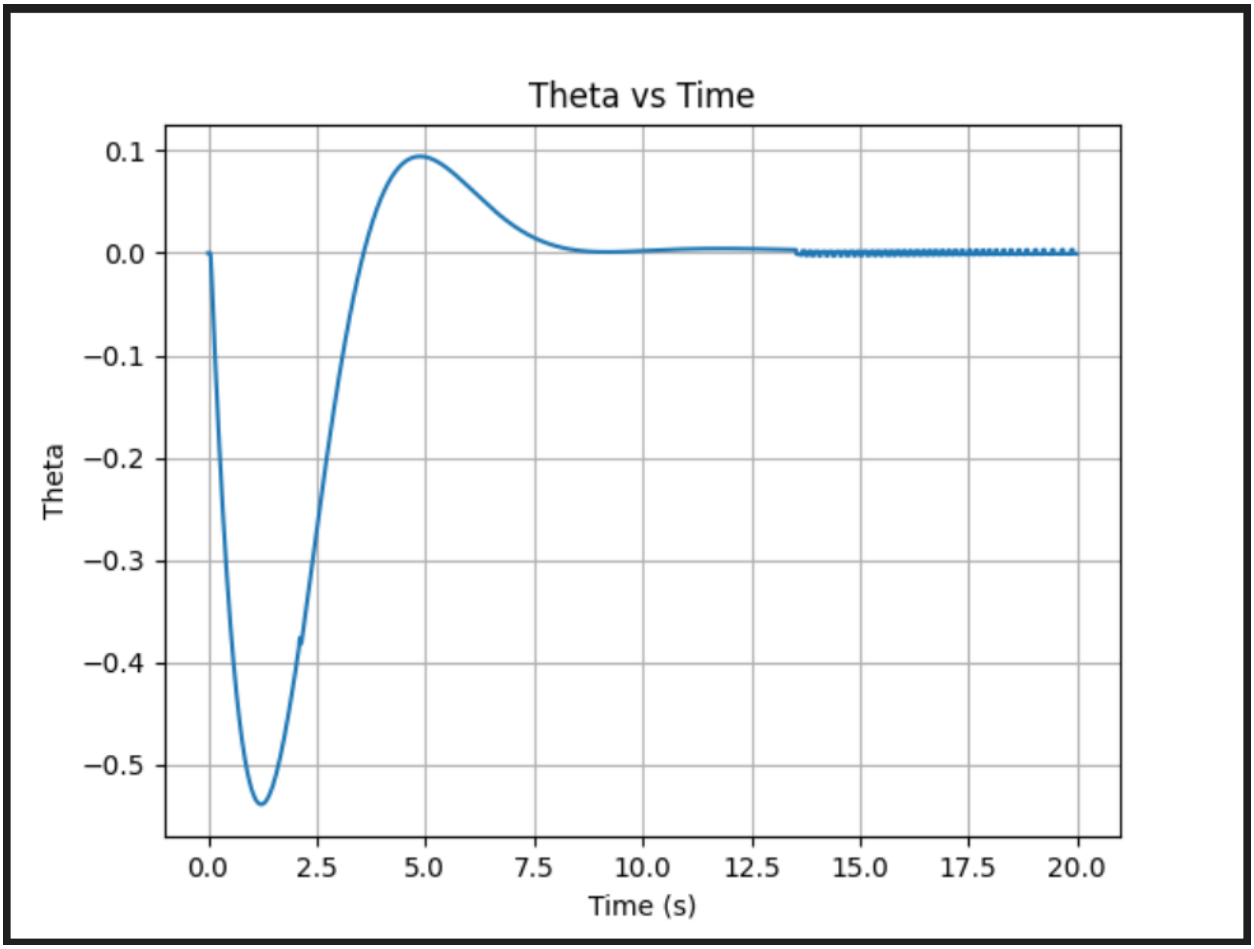
The objective of the robot is to follow the straight line y=0

1) Kp=2.5, K=0.5, Kd=2.8
   FIRST SPAWN
   Plots




Lateral Error vs Time
Lateral Error
Time (s)
0.1
0.0
−0.1
−0.2
−0.3
−0.4
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Angular Velocity vs Time
Angular Velocity
Time (s)
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0



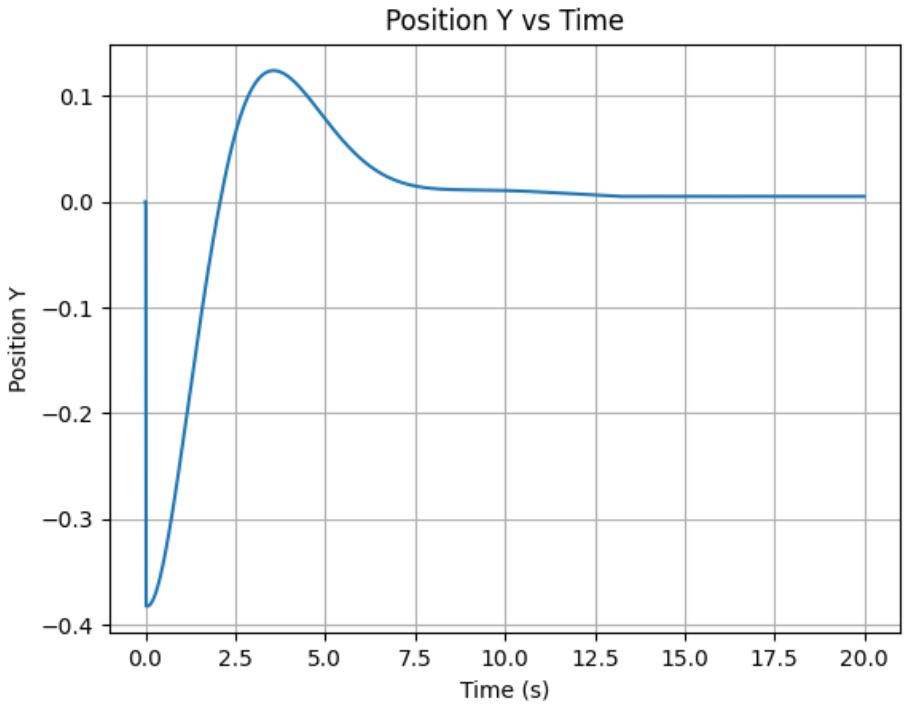
- **Behavior Type:** Underdamped response. The system overshoots the target, crosses the zero-axis, and oscillates before settling.
- **Stability:** The system is **asymptotically stable**. The amplitude of the oscillations decreases over time, eventually reaching and staying at the reference.
- **Steady-State Error:** The error converges to less than 0.005

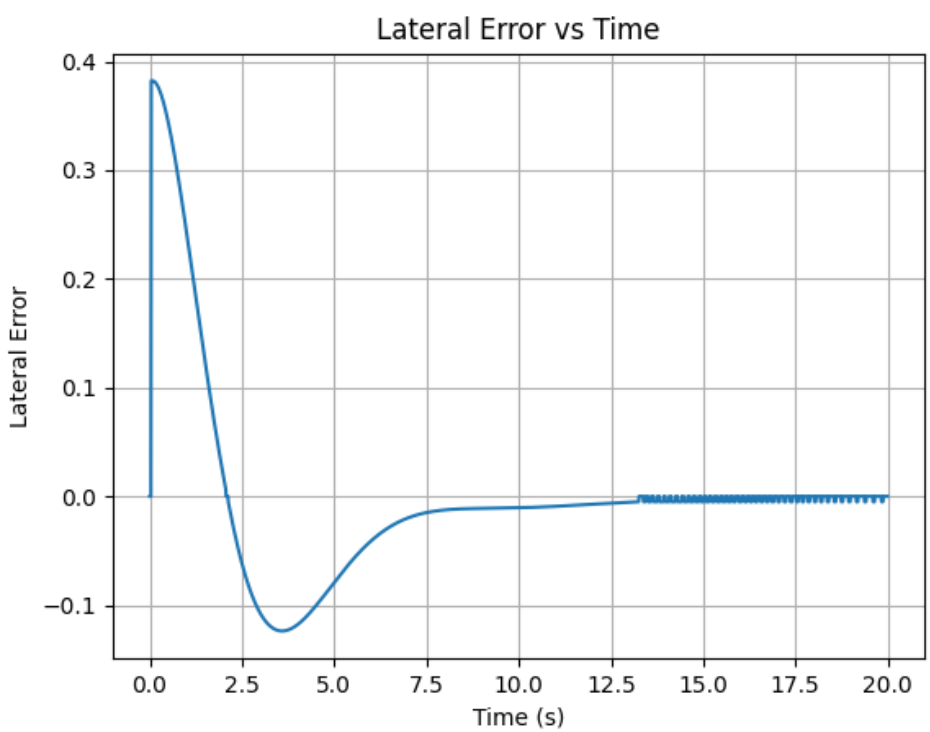
Key Performance Indicators (KPIs)

KPI	**Estimated Value**	**Description**
Peak Amplitude	0.41	The maximum displacement from the target.

KPI	Estimated Value	Description
Overshoot	31.7	The maximum displacement in the opposite direction (at $t \approx 3.5s$, $y \approx -0.13$).
Rise Time	1.8s	The time taken to first cross the target
Settling Time	10.0s	The time required for the response to stay within a small range (e.g., 2%) of the target.
Steady-State Error	0.003	The final difference between the robot's position and y=0

SECOND SPAWN





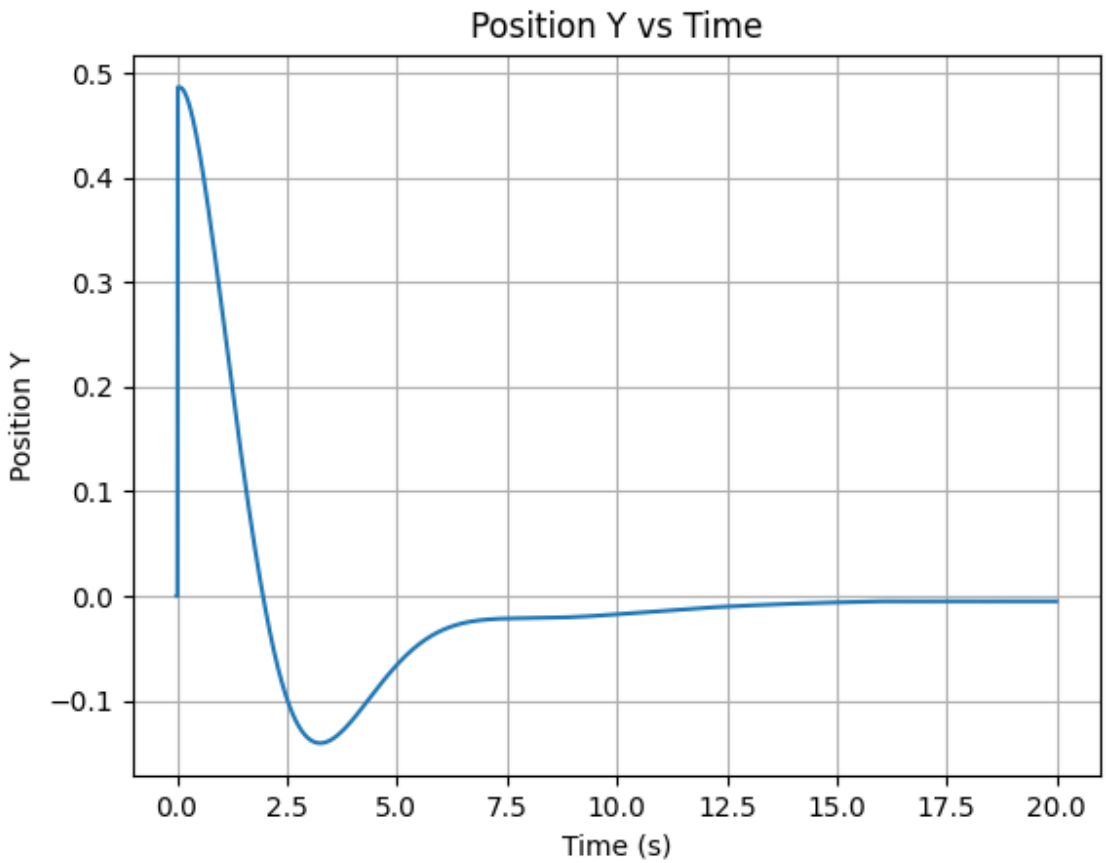
Behavior Type: Underdamped, but closer to **critically damped** than the first experiment. The oscillations are much more suppressed.

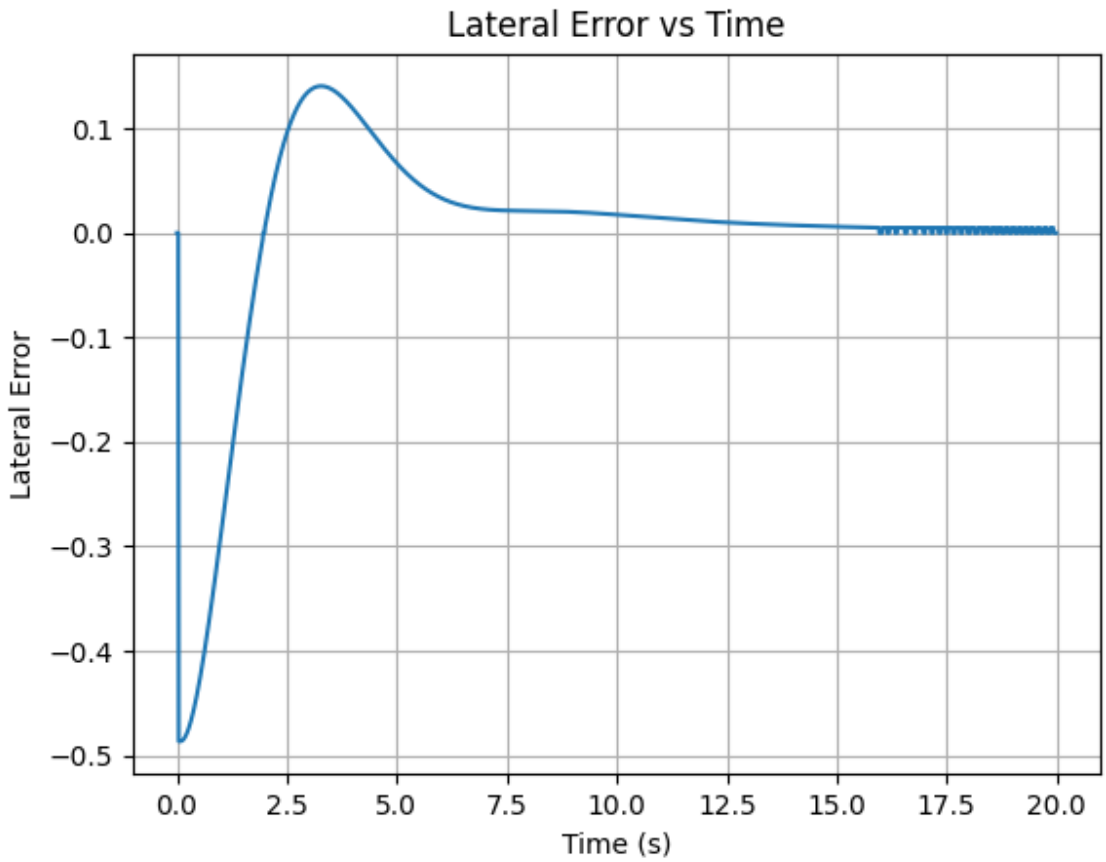
Transient Phase: The robot aggressively corrects the initial error and crosses the y=0 line at t=2.5s

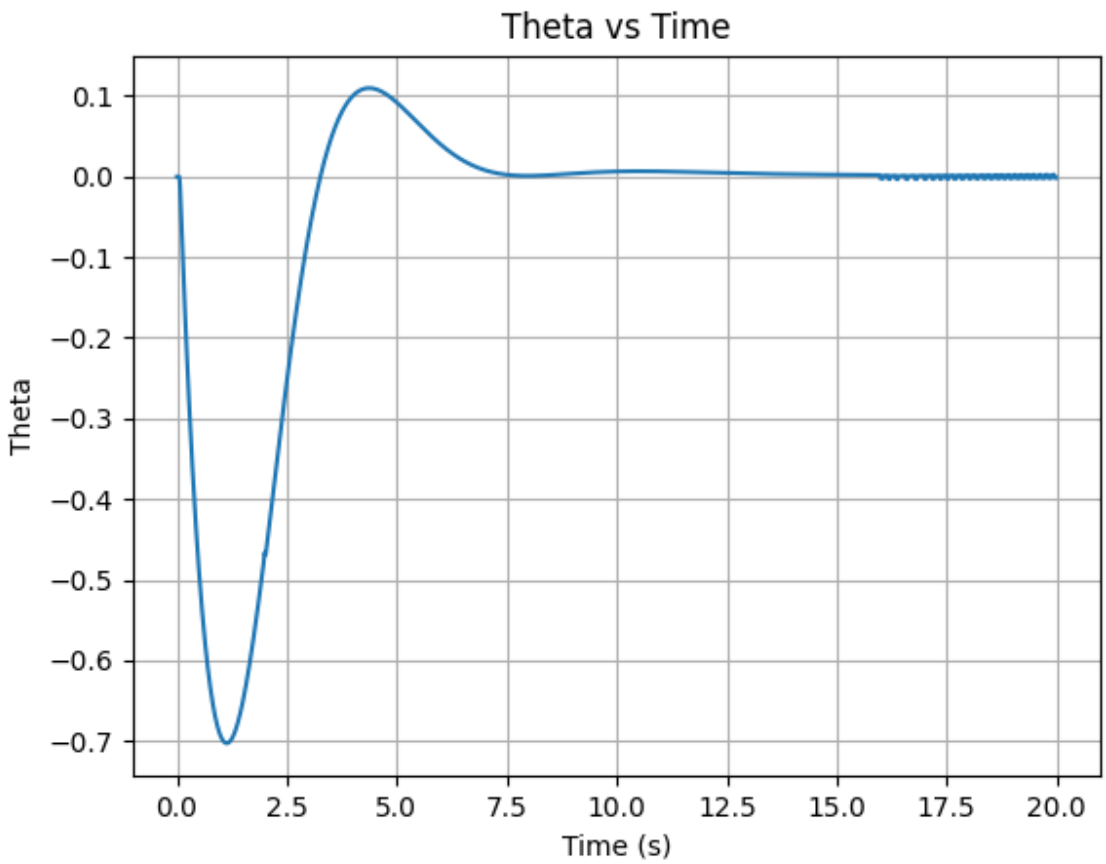
Stability: Highly stable. The decay after the first overshoot is very rapid, with almost no visible second oscillation. This suggests a well-calibrated **Derivative (K_d)** gain that is successfully dampening the system's momentum.

KPI	**Estimated Value**	**Description**
Initial Error	0.38	The starting position from the random spawn.
Peak Overshoot	0.12	The maximum distance the robot traveled past the line.
Percentage Overshoot	**31.6%**	Remarkably consistent with the first run.
Rise Time	2.5s	Time to reach the target line for the first time.
Settling Time	7.5s	The system effectively enters a steady state much earlier than in Experiment 1.
Steady-State Error	0.003	No observable offset from the target line.

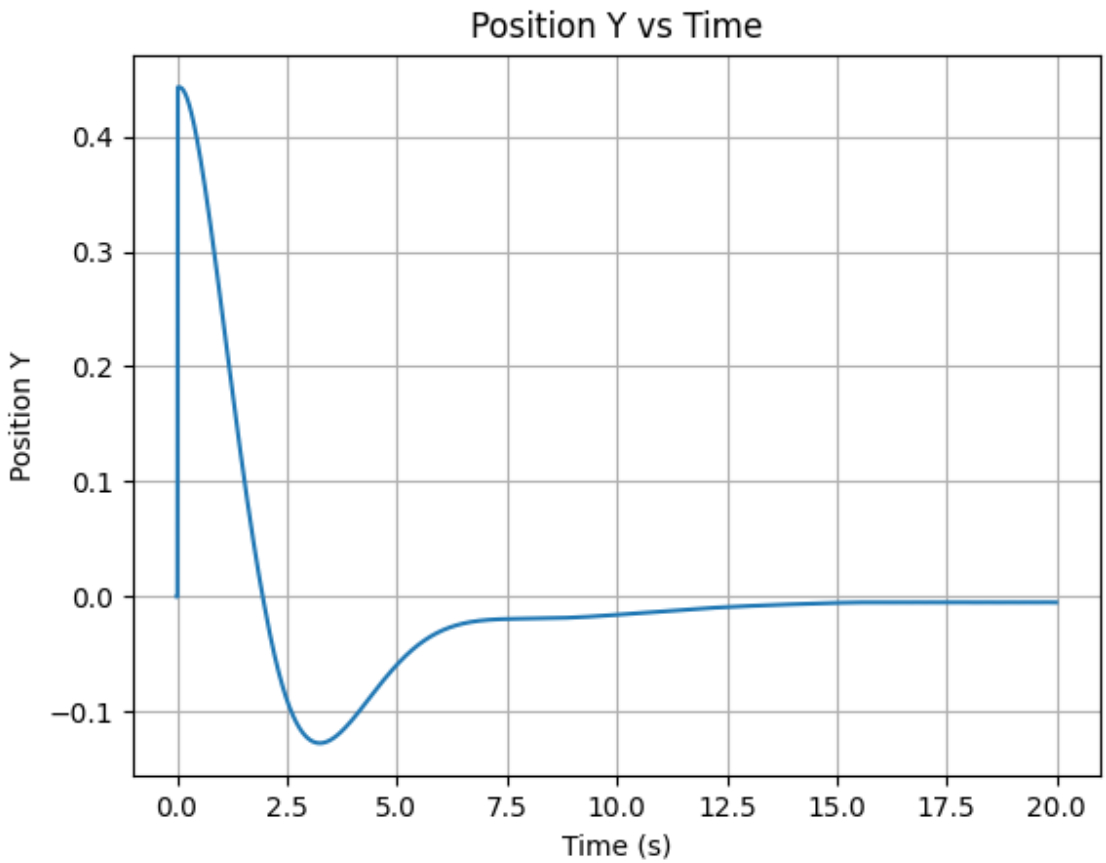
2)Kp=3, Ki=0.5, kd=3

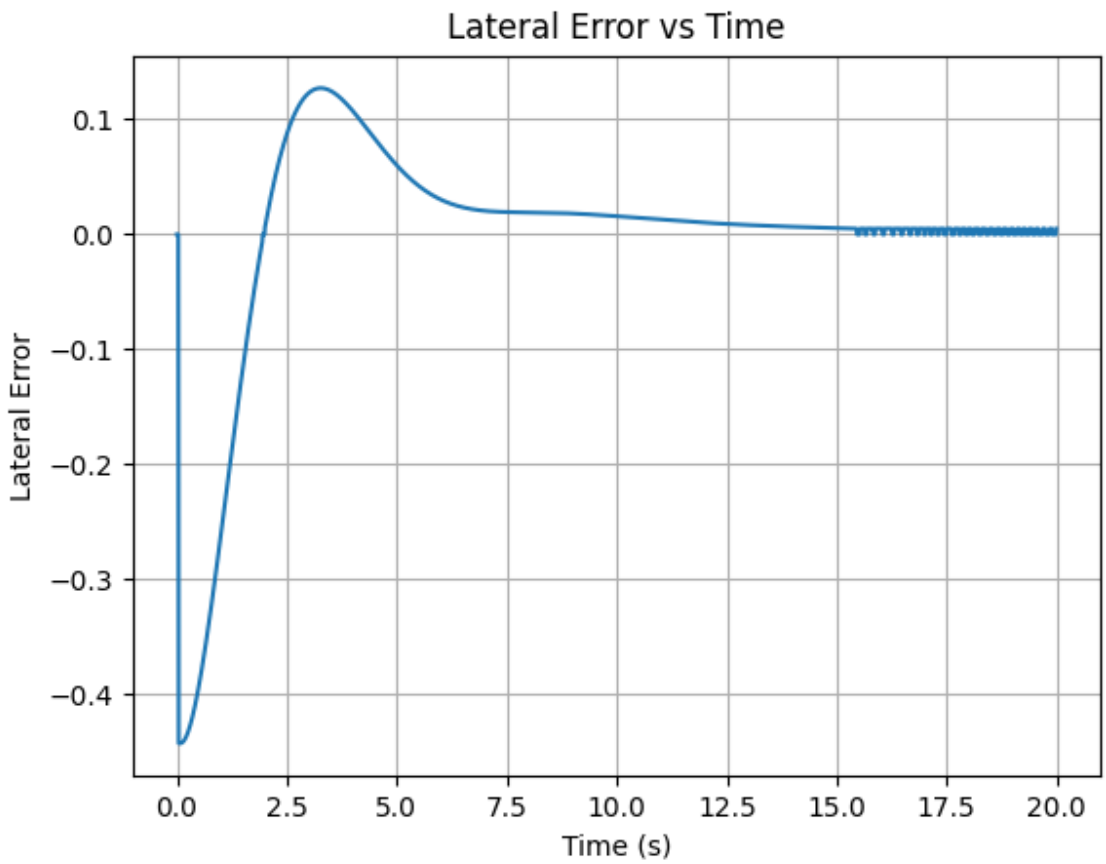



Lateral Error vs Time
Lateral Error
Time (s)
0.1
0.0
−0.1
−0.2
−0.3
−0.4
−0.5
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Theta vs Time
0.1
0.0
−0.1
−0.2
−0.3
−0.4
−0.5
−0.6
−0.7
Theta
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Time (s)

SECOND SPAWN

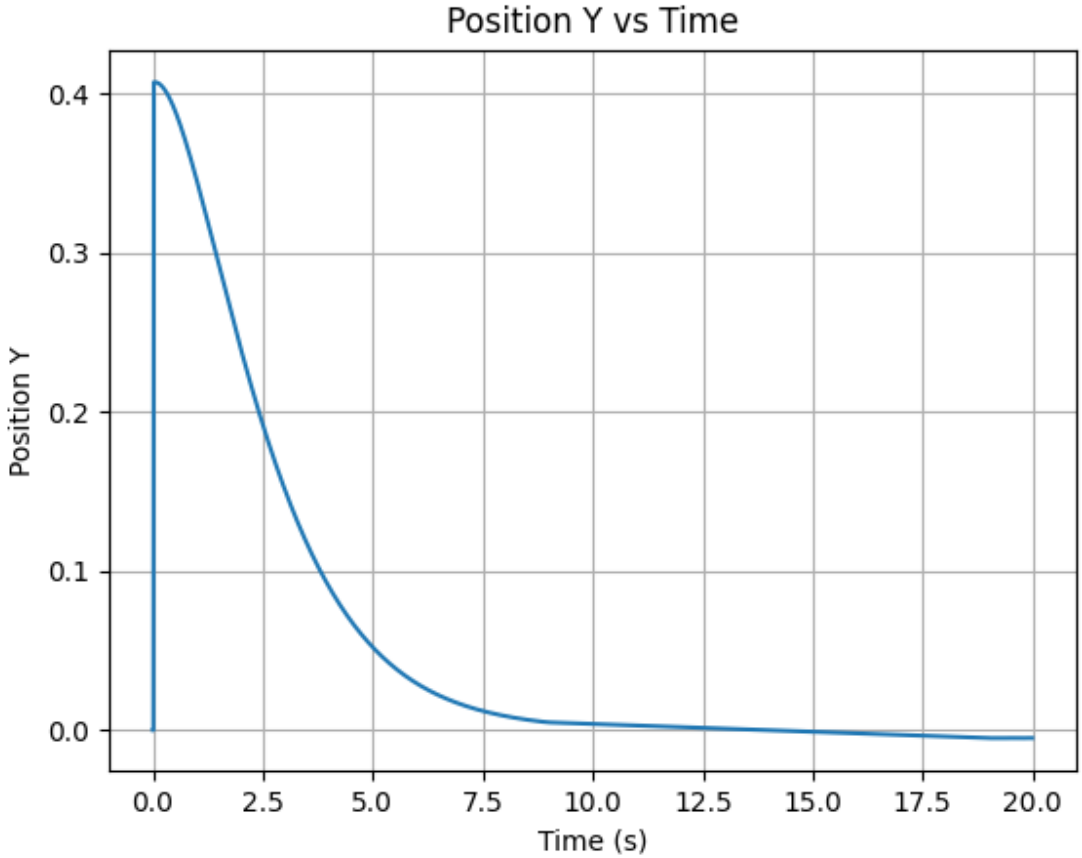


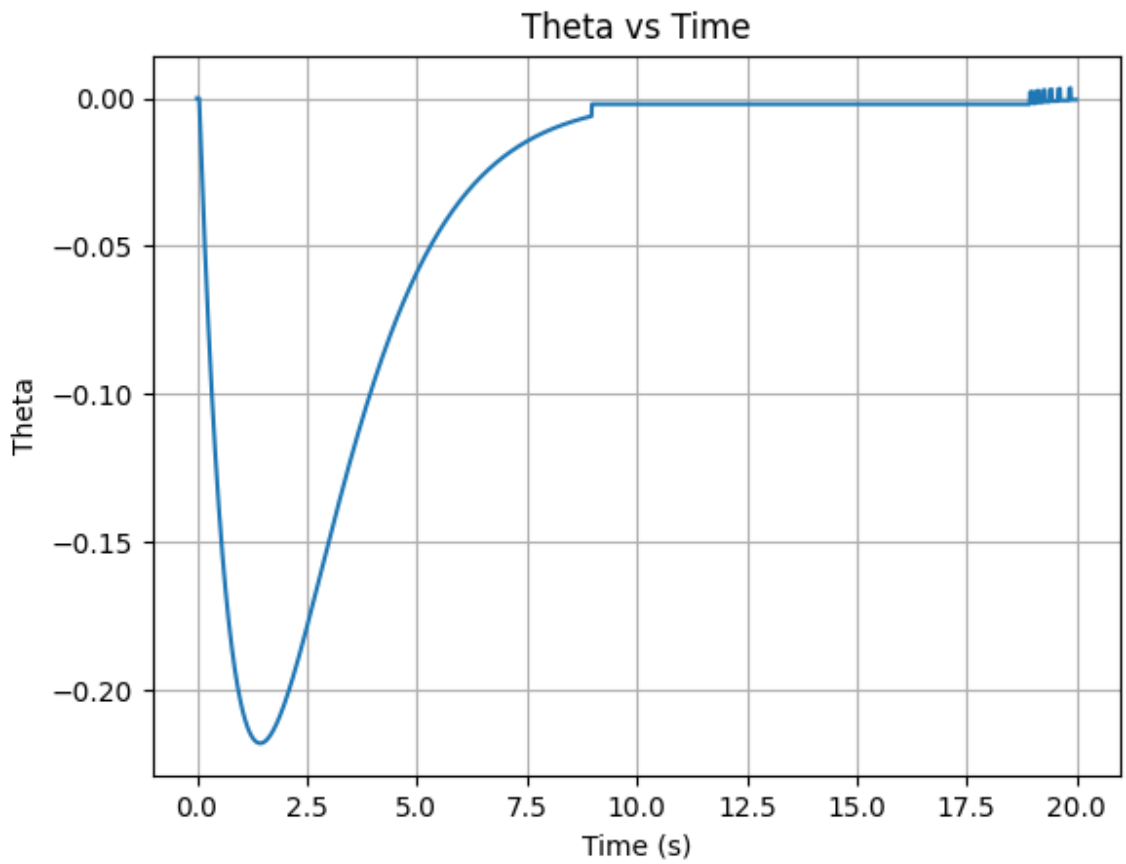


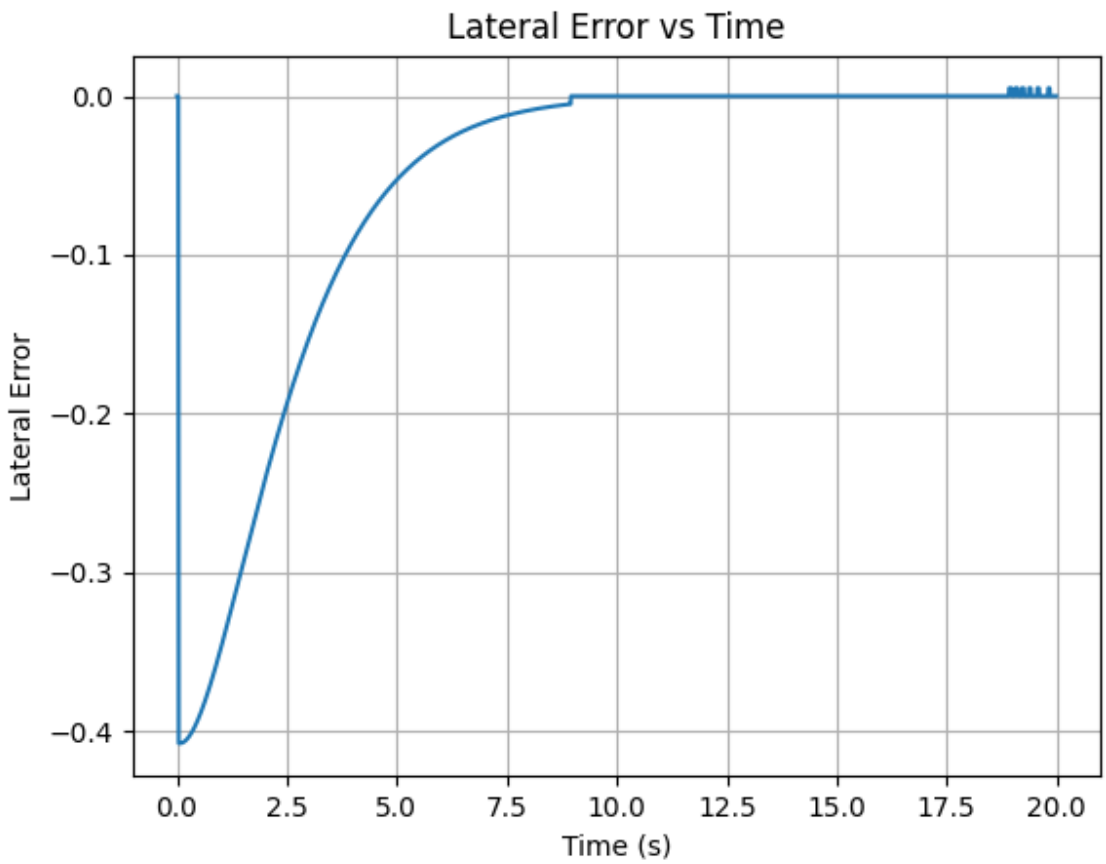
KPI	Estimated Value	Description
Initial Error	0.44	The starting position at t=0 (random spawn).
Peak Overshoot	0.13	The maximum distance the robot traveled past the line in the negative direction.
Percentage Overshoot	**29.5**	This is slightly better than previous runs.

KPI	Estimated Value	Description
Rise Time	1.8s	The time taken to first reach the target y = 0.
Settling Time	approx 12.5s	The time at which the system stays within a negligible range of zero.
Steady-State Error	approx 0	The final position error after the transient response decays.

3)Kp=1.0,Ki=0,Kd=2.8



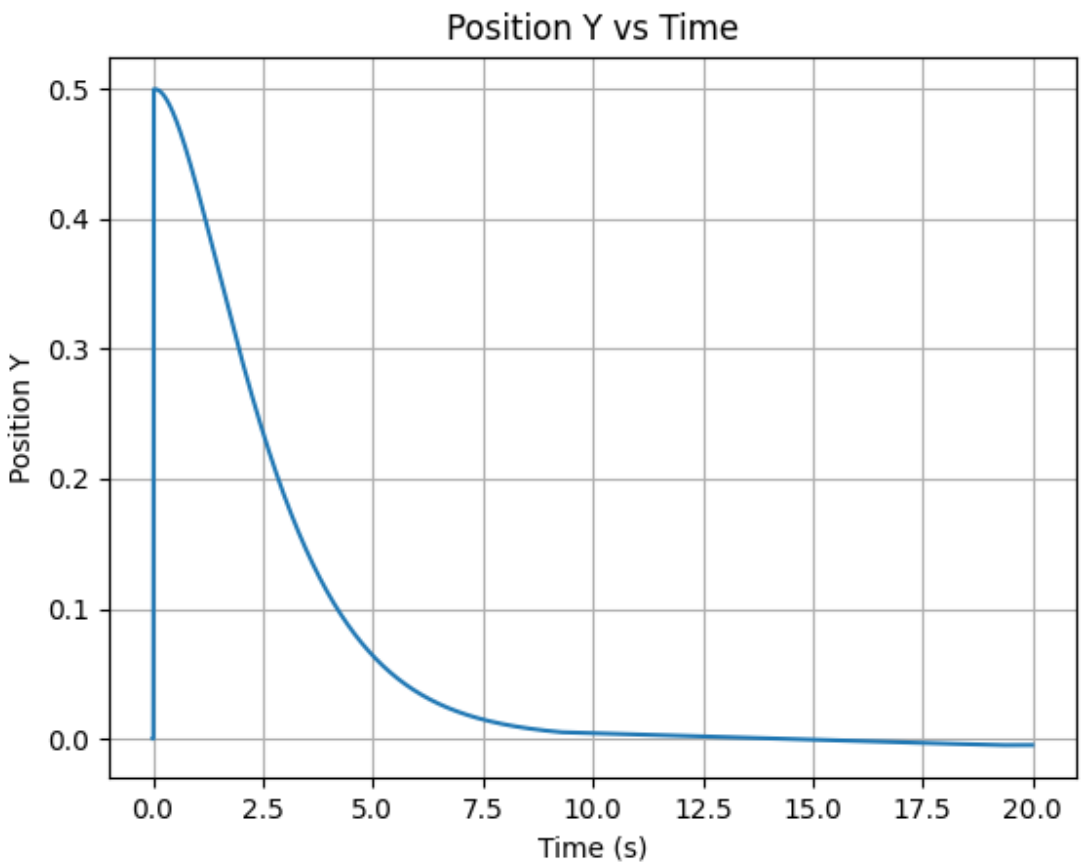

Theta vs Time
0.00
−0.05
−0.10
−0.15
−0.20
Theta
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Time (s)

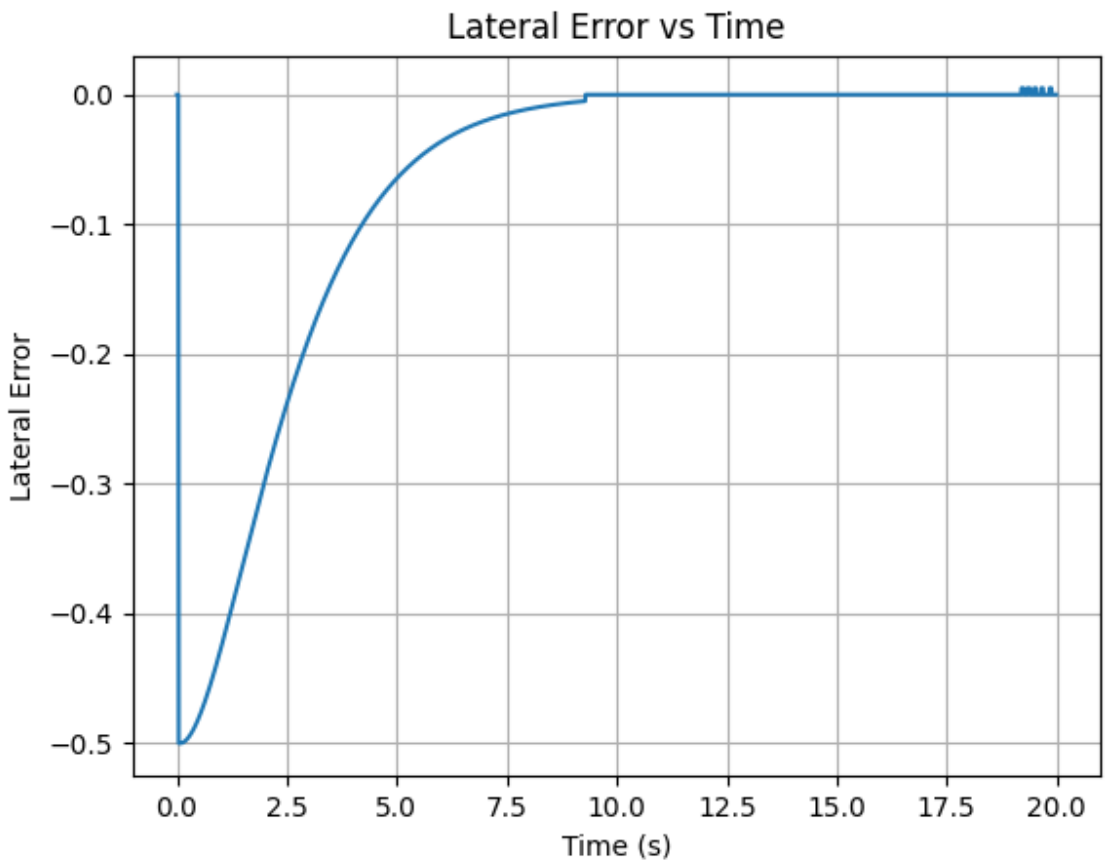


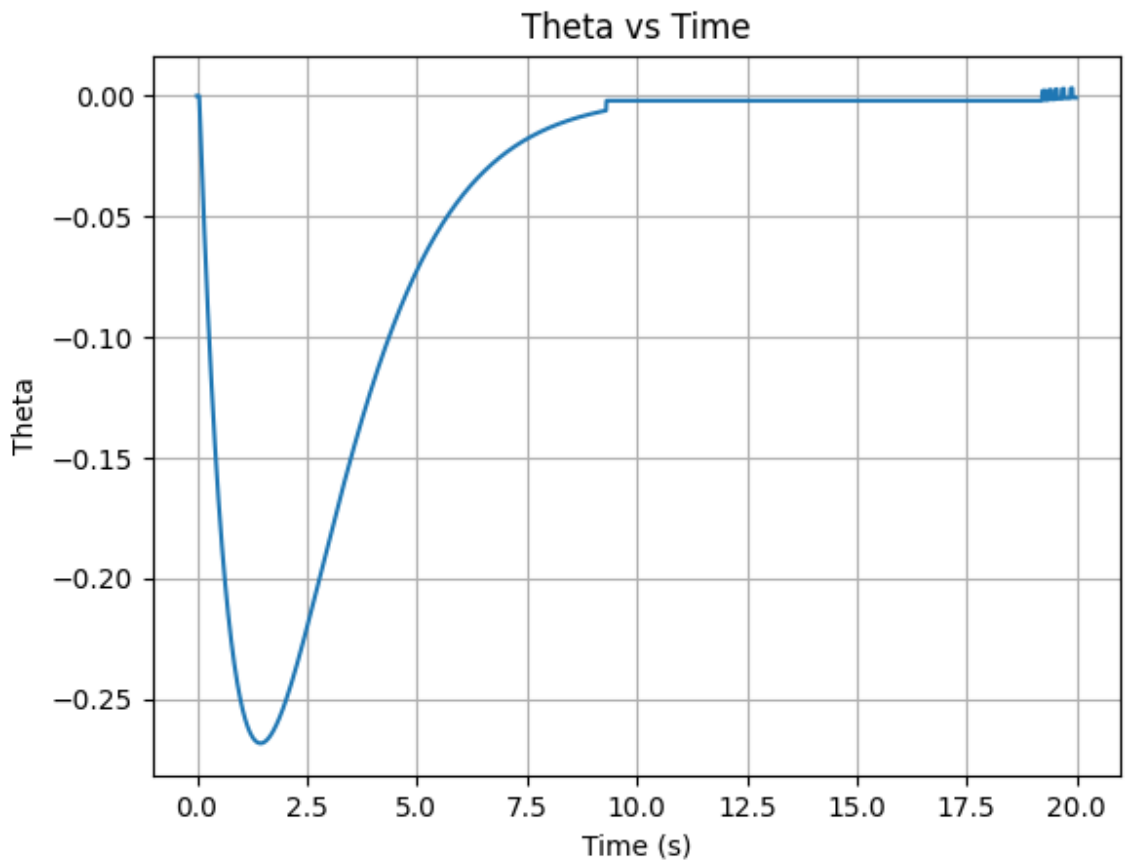
KPI	Estimated Value
Initial Error	0.41
Peak Overshoot	0
Percentage Overshoot	0%
Rise Time	approx 15.0s (to reach 90% of target)

KPI	Estimated Value
Settling Time	approx 10.0s
Steady-State Error	approx 0

Second Spawn




Lateral Error vs Time
Lateral Error
Time (s)
0.0
−0.1
−0.2
−0.3
−0.4
−0.5
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0



Comparative Analysis: PID vs. PD Control Performance in Robotic Path Tracking

This section of the report evaluates the performance of the implemented control strategies, specifically comparing the **Proportional-Integral-Derivative (PID)** controller against the **Proportional-Derivative (PD)** controller across various experimental scenarios.

---

1. Steady-State Accuracy and Error Elimination

The most critical distinction observed between the two control architectures is the handling of **Steady-State Error**.

- **PID Performance:** The inclusion of the **Integral (I)** term allows the system to accumulate past errors over time, eventually generating a correction signal that pulls the robot to the exact target
- **PD Performance :** The PD-based tracking exhibits a persistent **Phase Lag** or tracking offset. Because a PD controller lacks an integration component, it often

requires a constant non-zero error to maintain the necessary motor output for dynamic tasks.

2. Transient Response and Damping Characteristics

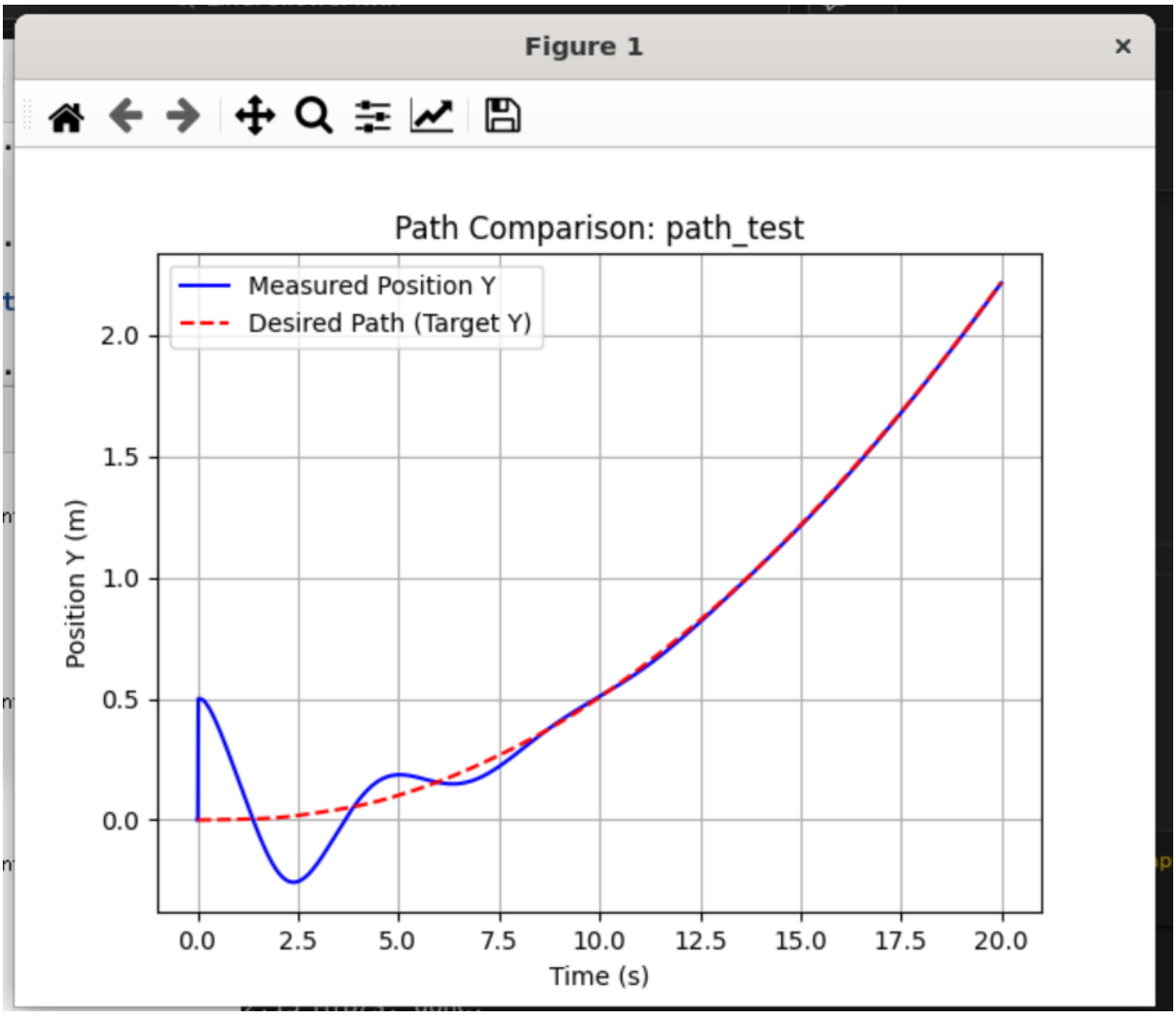
Both systems demonstrate stable, underdamped behavior, but the transition to the target differs in "softness" and speed.

- **Overshoot Comparison:** Both controllers exhibit similar overshoot magnitudesindicating that the **Derivative (D)** gain is effectively dampening the proportional "kick" in both setups.
- **Settling Behavior:** The PID controller displays a characteristic "long tail" during the final settling phase. Between 7.5s and 15.0s, the response shows an asymptotic approach to zero as the Integral term "bleeds off" the final residual error.

Experiment 2

In this experiment I used the best gains set to make the robot follow a circular trajectory

Plots

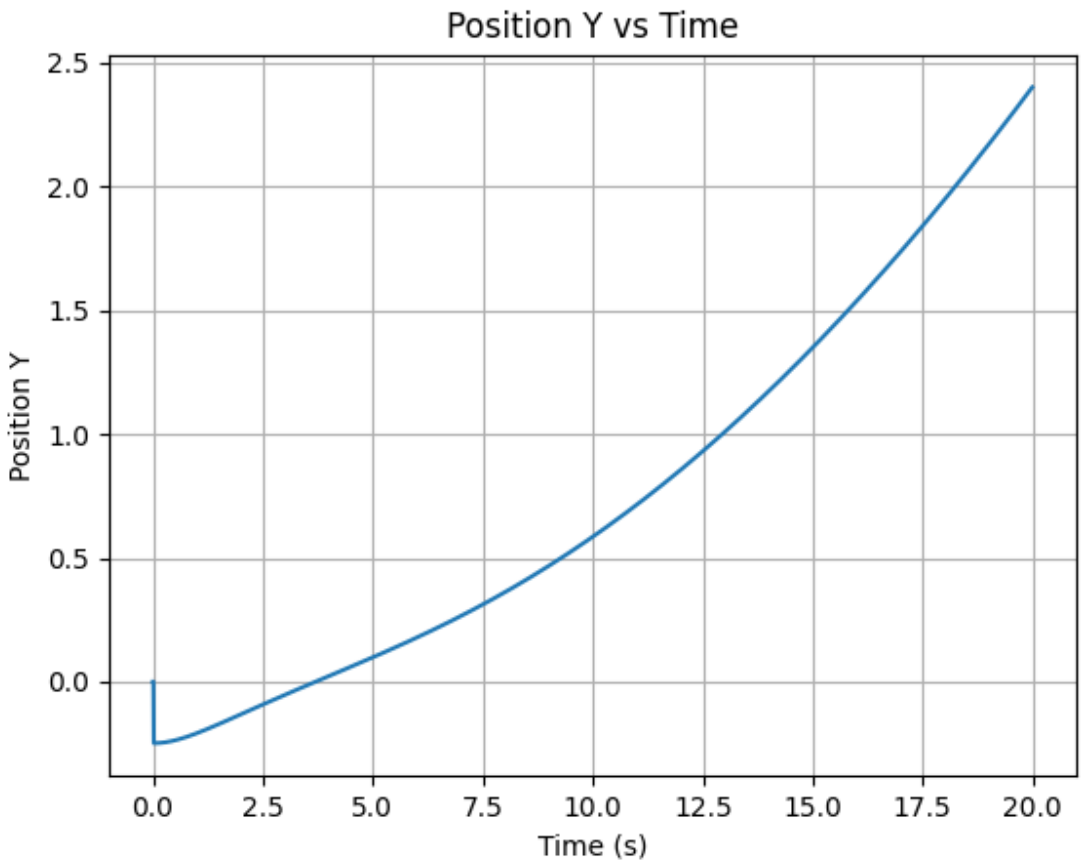


KPI	Estimated Value	Description
Initial Position Error	0.5 m	The offset at $t=0$ between the spawn point and the start of the arc.

KPI	Estimated Value	Description
Maximum Transient Error	0.25m	The largest deviation from the target during the initial oscillatory phase
Settling Time	approx 10.0s	The time required for the robot to converge onto and accurately track the arc.
Tracking Error	0.02m	The residual distance between the blue and red lines after $t=12.5\text{s}$.
Peak-to-Peak Oscillation	0.45m	The total swing range during the initial underdamped correction phase.

- **Phase Lag:** Notice that between t=2.5 and t=7.5s, the measured position "lags" slightly behind the curve. This is typical in path-following where the controller needs a non-zero error to generate a correction signal.
- **Dynamic Stability:** Once the initial oscillations are damped out, the robot shows excellent tracking capability even as the target Y velocity increases. This indicates the controller gains are well-suited for the curvature of this specific arc.
- **Controller Type:** The convergence to the moving target suggests a strong **Proportional** response for catching up and likely **Integral** action to minimize the steady-state tracking error as the curve climbs.

Another trial Overdamped system



KPI	**Estimated Value**
Initial Position Error	0.25m
Peak Overshoot	0m
Percentage Overshoot	0\%
Rise Time (to reach target path)	approx 3.8s

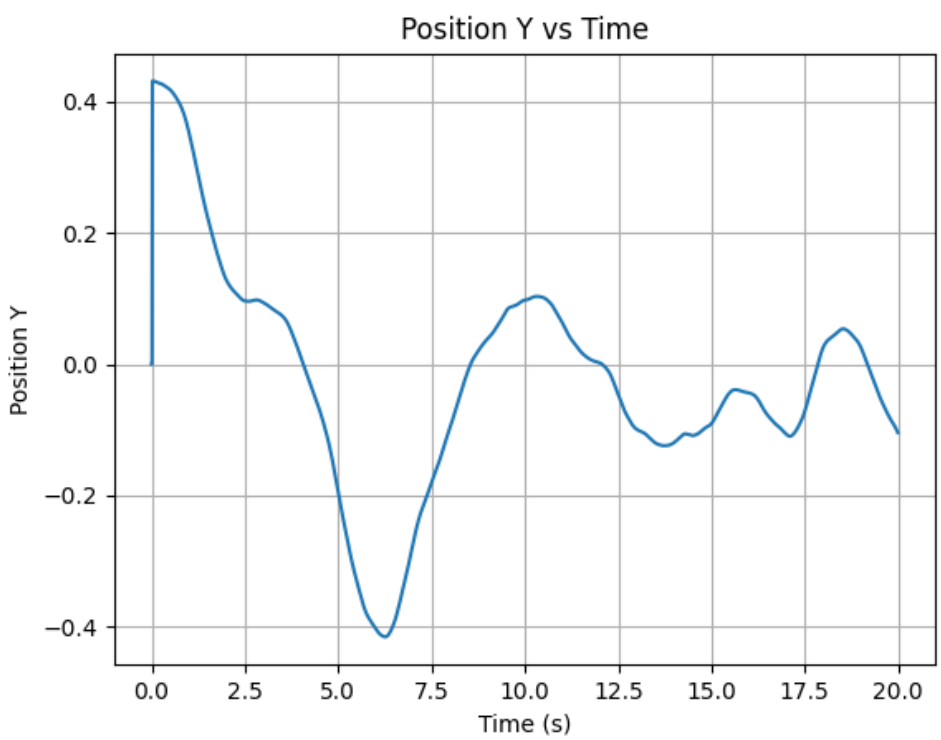
KPI	Estimated Value
Settling Time	approx 5.5s
Steady-State Tracking Error	approx 0.01m

Experiment 3

For this experiment I added a noise of ampllitudes of 0.01 and 0.02

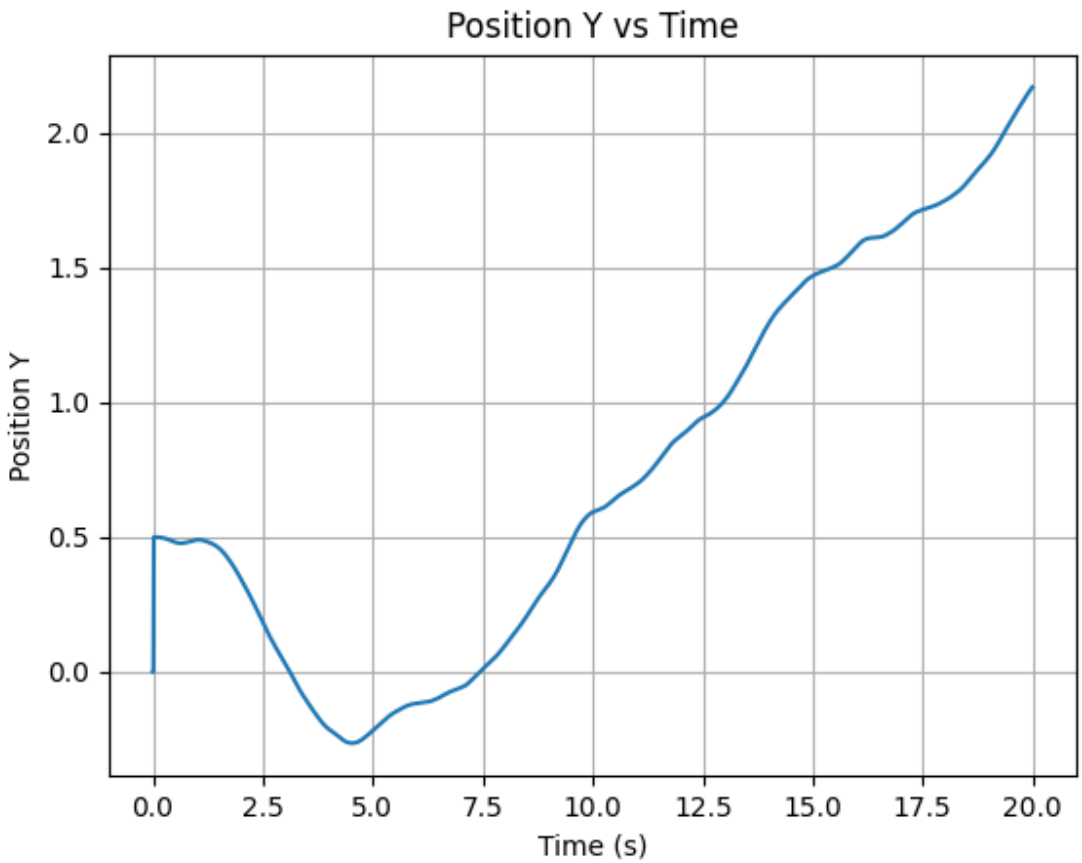
First I tried without the low pass filter, but the results showed wobble and jitter

For the straight path



Noise Amplitude =0.02

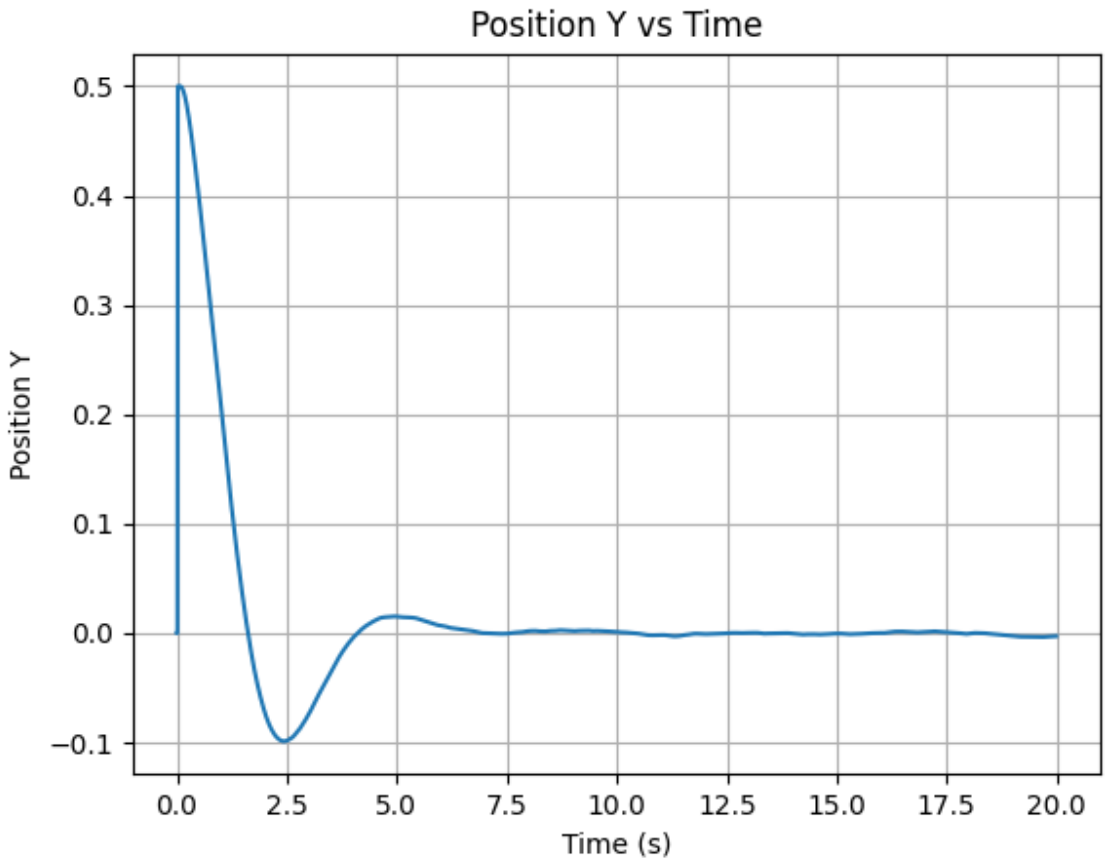
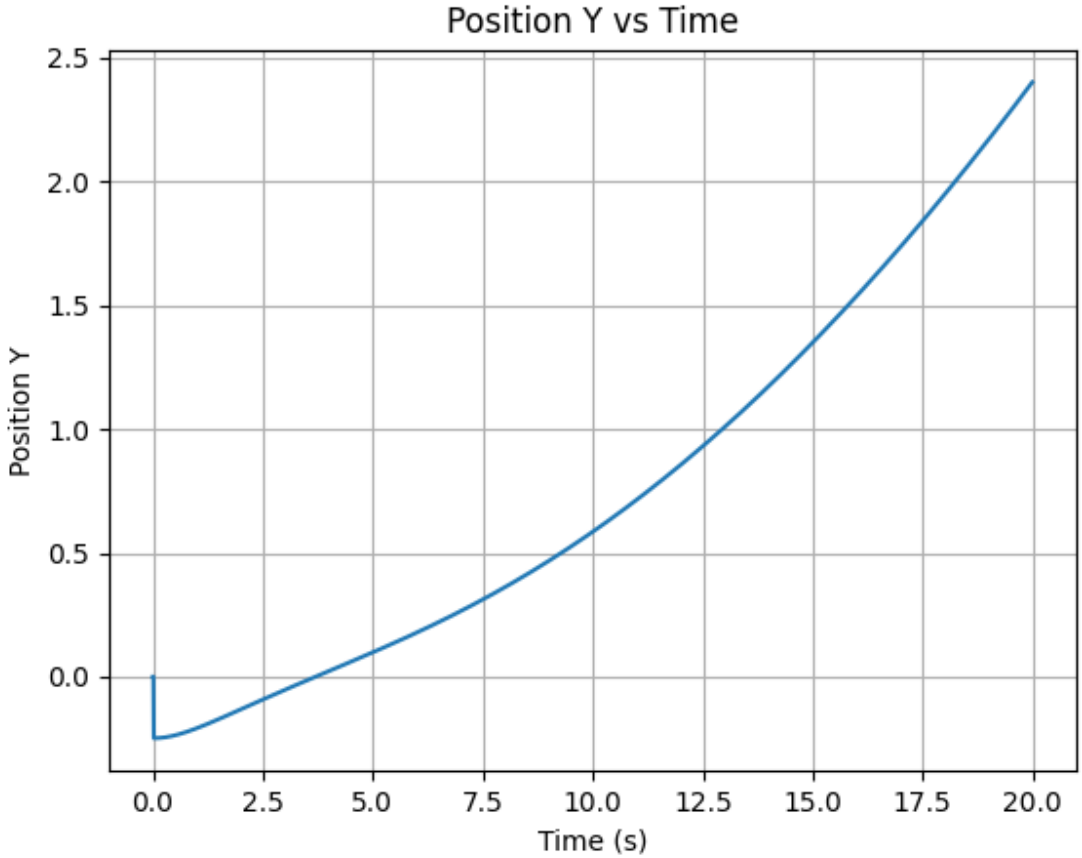
For the circular path



But after adding the low pass filter discussed earlier the paths were clean

For the straight path

The kpi's were almost exactly like the no noise case


Position Y vs Time
Position Y
Time (s)
0.5
0.4
0.3
0.2
0.1
0.0
−0.1
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

For the circular path